Horizontal to Vertical Privilege Escalation Master Cheatsheet

OWASP A01 + A05

Bridging lateral access flaws to administrative domain takeover & high-privilege abuse.

1 Concept & Transition Architecture

Horizontal to Vertical Escalation: The exploitation chain where an attacker starts with standard user access (User A), leverages a **Horizontal flaw (IDOR / BOLA)** to compromise a peer user account (User B, Moderator, or Manager), and subsequently utilizes administrative functions or hidden role-flags exposed within User B's context to achieve **Vertical Escalation (Admin Takeover)**.

2 Common Transition Vectors & Entry Points

Vector Type	Mechanism	Impact & Outcome
IDOR to Password Reset	Swapping `user_id` in password reset request to trigger token generation for admin accounts.	Account takeover of high-privilege users.
Profile Update & Role Injection	Injecting `"role": "admin"` or `"is_admin": true` during standard profile updates.	Privilege escalation via Mass Assignment.
Session / Token Hijacking	Accessing another user's session token, API key, or JWT stored in an accessible profile object.	Direct administrative session inheritance.
Hidden Admin Endpoints	Discovering administrative paths or management parameters inside normal user objects or responses.	Unauthenticated control panel access.

3 Step-by-Step Escalation Methodology

1. **Reconnaissance & User Enumeration:** Identify all user accounts, roles, and identifier structures (sequential or UUID) within the application.
2. **Horizontal Compromise:** Locate an IDOR or BOLA endpoint that allows viewing or modifying administrative or higher-privileged user metadata.
3. **Privilege Pivot:** Exploit parameter tampering, role flags, or token leakage to elevate the compromised peer account's privileges.

4 Practical Scenarios & Code Exploits

A. Mass Assignment Role Injection

Modifying user update payloads to append administrative roles directly into database binding.

B. Password Reset IDOR Pivot

Targeting admin account identifiers in reset forms to intercept reset links.

Vulnerable API Request (Mass Assignment Role Escalation):

```
PUT /api/v1/user/update_profile HTTP/1.1 Host: target-app.com Authorization: Bearer USER_JWT_TOKEN Content-Type: application/json { "user_id": 105, "first_name": "John", "email": "user@example.com", "is_admin": true, "role": "administrator" }
```

5 Secure Coding & Remediation

❌ **Vulnerable Code (Implicit Binding / Mass Assignment):**

```
@PostMapping("/api/user/update") public User updateUser(@RequestBody User updatedUser, HttpSession session) { // Unsafe: Binds all incoming JSON fields directly to entity, allowing role tampering return userRepository.save(updatedUser); }
```

✅ **Secured Code (Explicit Field Filtering & RBAC Validation):**

```
@PostMapping("/api/user/update") public ResponseEntity<String> updateUser(@RequestBody UserDto dto, HttpSession session) { User currentUser = (User) session.getAttribute("currentUser"); // Explicitly prevent role modification by standard users if (dto.getRole() != null && !currentUser.hasRole("ADMIN")) { throw new AccessDeniedException("Unauthorized role modification attempt."); } return ResponseEntity.ok("Profile updated securely."); }
```

6 Verification & Testing Checklist

- [ ] Tested if standard user update endpoints accept restricted role parameters (e.g., `role=admin`).
- [ ] Checked IDOR vulnerabilities on password reset or profile management endpoints for high-privilege users.
- [ ] Verified server-side role validation on all state-changing API operations.